Department of Computer Science & Engineering

Microprocessor & Computer Architecture
UE24CS251B

MPCA Orange Problem

Name: Tanmayi Nagabhairava	SRN: PES1UG24CS493	Section: I

A

Question A:

Code with comments:

```
.data
   signal:     .word 5, -3, 12, 7, 9, 0, -2, 15    # Given data
   n:          .word 8                             # N=8
   stable_cnt: .word 0
   net_dev:    .word 0
   drop_cnt:   .word 0
   spike_idx:  .word -1

.text
main:
   la   x10, signal       # Load base address
   lw   x11, n            # Load N
   li   x12, 0            # Loop counter (i)
   li   x13, 0            # stable_cnt accumulator
   li   x14, 0            # net_dev accumulator
   li   x15, 0            # drop_cnt accumulator
   li   x16, -1           # spike_idx (default -1)
   li   x17, 0            # Found spike flag (0 = no, 1 = yes)

loop:
   beq  x12, x11, end_loop # If i == N, exit

   slli x18, x12, 2       # Offset = i * 4
   add  x18, x18, x10     # Address of signal[i]
   lw   x19, 0(x18)       # x19 = current value

   # 1. Stability Check (-5 to +5)
   li   x20, -5
   li   x21, 5
   blt  x19, x20, check_net
   bge  x21, x19, is_stable
   j    check_net
is_stable:
   addi x13, x13, 1

check_net:
```

```
    # 2. Net Deviation & 3. Drop Events
    blt  x19, x0, is_negative
    add  x14, x14, x19      # Sum positive values
    j    check_spike
is_negative:
    sub  x14, x14, x19      # Subtract negative values (Net = Pos - Neg)
    addi x15, x15, 1        # Increment drop count

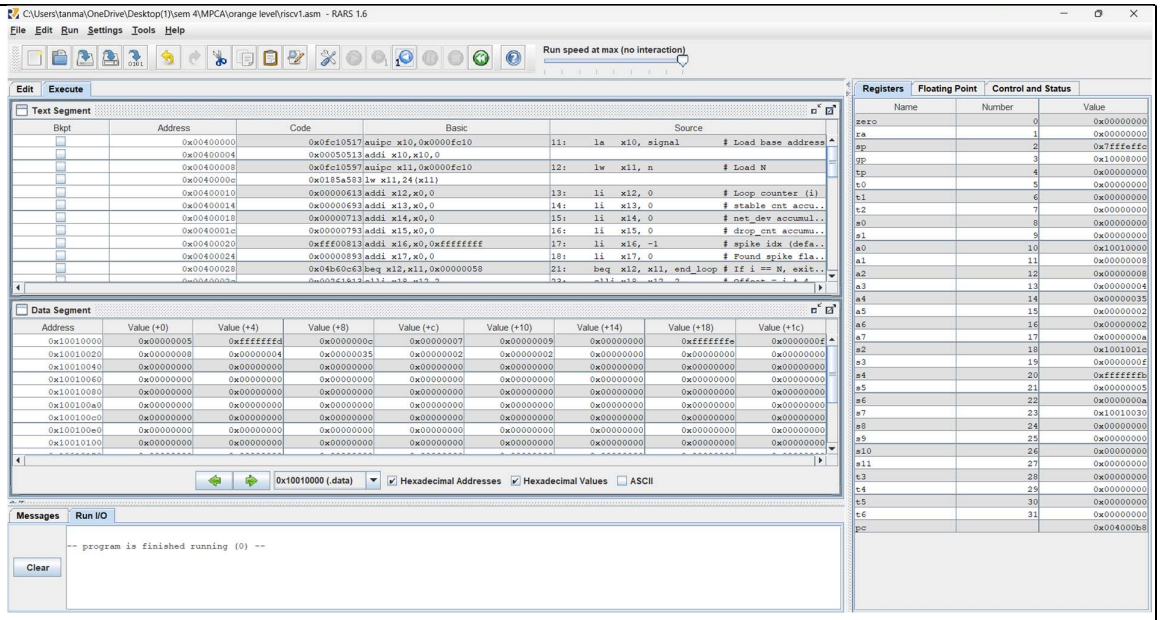
check_spike:
    # 4. First Spike Index (> 10)
    li   x22, 10
    ble  x19, x22, next_iter
    bne  x17, x0, next_iter # If already found, don't update
    addi x16, x12, 0        # Store current index
    li   x17, 1             # Set found flag

next_iter:
    addi x12, x12, 1        # i++
    j    loop

end_loop:
    # Store results in memory
    la   x23, stable_cnt
    sw   x13, 0(x23)
    la   x23, net_dev
    sw   x14, 0(x23)
    la   x23, drop_cnt
    sw   x15, 0(x23)
    la   x23, spike_idx
    sw   x16, 0(x23)

    # Exit program
    li   a7, 10
    ecall
```

Screenshot of the output:



Explanation of code:

The program starts by loading the base address of the signal array and the value N=8 into the register. It sets the accumulators for the stable count, net deviation, and signal drop count while setting the default spike index to -1. It then goes through the loop that iterates through the array using pointer arithmetic. It checks each value in the array to determine if the value is within the stable range of -5 to +5. It branches based on the condition and adds the value if it is within the stable range. It adds positive values to the accumulator while subtracting the negative values from the accumulator to determine the net deviation. It has a counter for signal drop if the value is less than zero. It then goes through the array to determine the first index of the signal spike if the value is greater than 10. It stores the index while avoiding pseudo-instruction usage and stores the results in the memory.

B

Question B:

Code with comments:

```
.data
   n_val:    .word 7        # Input value N
   is_even:  .word 0        # Result: 1 if even
   is_prime: .word 0        # Result: 1 if prime
   is_fact:  .word 0        # Result: 1 if factorial

.text
main:
   lw   x10, n_val          # Load N into x10

   # --- 1. Even/Odd Check ---
   andi x11, x10, 1         # Extract LSB; if LSB is 0, number is even
   beq  x11, x0, set_even
   li   x12, 0              # It is odd
   j    prime_check
set_even:
```

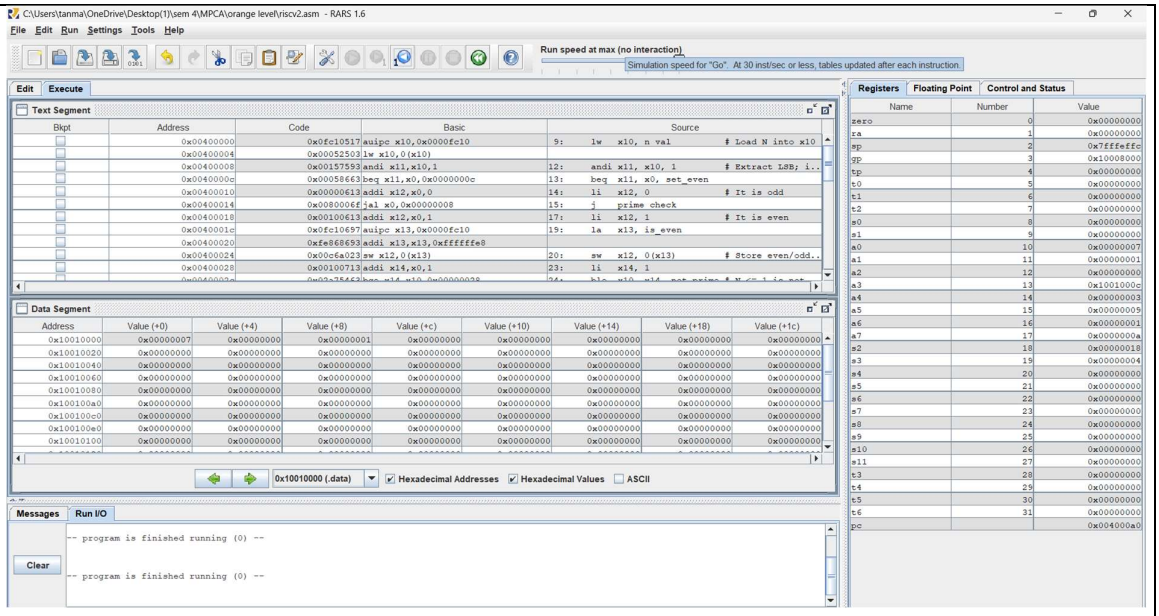
```
    li   x12, 1             # It is even
prime_check:
    la   x13, is_even
    sw   x12, 0(x13)        # Store even/odd result

    # --- 2. Prime Check ---
    li   x14, 1
     ble  x10, x14, not_prime # N <= 1 is not prime
    li   x14, 2             # Divisor d = 2
prime_loop:
    mul  x15, x14, x14      # x15 = d * d
    blt  x10, x15, set_prime # If N < d*d, then N is prime
    rem  x16, x10, x14      # x16 = N % d
    beq  x16, x0, not_prime  # If N % d == 0, not prime
    addi x14, x14, 1        # d++
    j    prime_loop
set_prime:
    li   x17, 1
    j    store_prime
not_prime:
    li   x17, 0
store_prime:
     la   x13, is_prime
    sw   x17, 0(x13)

    # --- 3. Factorial Check ---
    li   x18, 1             # Current factorial product
    li   x19, 1             # Multiplier counter
fact_loop:
    beq  x18, x10, set_fact  # If product == N, N is a factorial
    blt  x10, x18, not_fact  # If product > N, N is not a factorial
    addi x19, x19, 1        # counter++
    mul  x18, x18, x19      # product = product * counter
    j    fact_loop
set_fact:
    li   x20, 1
    j    store_fact
not_fact:
    li   x20, 0
store_fact:
    la   x13, is_fact
    sw   x20, 0(x13)

    li   a7, 10             # Exit
    ecall
```

Screenshot of the output:



Explanation of code:

This program will check three properties of the positive integer N stored in the memory. It will check if the number is even or odd by using the bitwise AND instruction with the number 1. If the result is 0, the number is even. It will check if the number is prime by looping through the division of the number by integers from 2 up to the square root of the number. If the remainder is 0 at any point during the division, the number is not prime. It will check if the number is a factorial number by looping through the multiplication of consecutive integers from 1 through the number. If the result is equal to the number, the number is a factorial number; if the result is larger than the number, the number is not a factorial number. This program is designed to be compatible with RARS using only RISC-V arithmetic and branch instructions without the use of pseudo-instructions.